REAL-TIME 3D AVATAR SYNTHESIS FOR INDIAN SIGN LANGUAGE

A PROJECT REPORT

Submitted by

RANJITH R

(810424622017)

In partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

FACULTY OF COMPUTER SCIENCE AND INFORMATION ENGINEERING

DHANALAKSHMI SRINIVASAN ENGINEERING COLLEGE

(Autonomous)

PERAMBALUR – 621 212

MAY-2026

DHANALAKSHMI SRINIVASAN ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

PERAMBALUR – 621 212

BONAFIDE CERTIFICATE

Certified that this Project report **ON REAL-TIME 3D AVATAR SYNTHESIS FOR INDIAN SIGN LANGUAGE"** is the Bonafide work of **RANJITH R (810424622017)** who carried out the research under my supervision.

SIGNATURE
Prof. D.VIJAYAKUMAR
HEAD OF THE DEPARTMENT,
Department of MCA&CST,
Dhanalakshmi Srinivasan
Engineering College(A),
Perambalur-621212

SIGNATURE
Ms. KALPANA.A
Asst Prof SUPERVISOR,
Department of MCA,
Dhanalakshmi Srinivasan
Engineering College(A),
Perambalur-621212

Submitted for the Project Viva-Voce Examination held on_____

INTERNAL EXAMINER **EXTERNAL EXAMINE**

ACKNOWLEDGEMENT

First and foremost, I am deeply grateful to GOD ALMIGHTY for his abundant blessing, guidance and strength throughout the course of this project.

I express my special privilege and heartfelt to Shri. A. SRINIVASAN, Chancellor of Dhanalakshmi Srinivasan University, Perambalur.

I express my special thanks to Dr.D.SHANMUGASUNDARAM M.E.,Ph.D.,F.I.E.,C.Eng., Principal of Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur for his valuable contribution for study.

I would like to express my sincere gratitude to Dr.K. ANBARASAN M.E,Ph.D, Vice principal of Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur ,and also sincere thanks to Dr.M.CHELLAPPAN M.E,Ph.D, Controller of Examinations, Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur for his valuable support .

I would like to express my sincere gratitude to Prof. D.VIJAYAKUMAR , M.Tech (Ph.D), Head of the Department of Master of Computer Applications, Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur for his guidance and encouragement.

I am greatly thanks to Ms. KALPANA.A, MCA, Asst Prof., Department of Master of Computer Applications, Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur, My Supervisor for extending constant support and valuable guidance for the success completion of Project work.

I would like to extend my thanks to all the faculty members of the Department of Master of Computer Applications, Dhanalakshmi Srinivasan Engineering College (Autonomous), Perambalur, who had helped me a lot to complete this project successfully.

Finally, I would like to thank my family and friends for their constant encouragement and understanding during this time.

RANJITH R

REAL-TIME 3D AVATAR SYNTHESIS FOR INDIAN SIGN LANGUAGE

S.NO	TOPIC/TITLE	PAGE
	ACKNOWLEDGEMENT	**I**
	ABSTRACT	**IV**
1	INTRODUCTION 1.1 Overview Of The Project 1.2 Problem Description	**1**
2	SYSTEM ANALYSIS 2.1. Existing System 2.2. Proposed System	**4**
3	SYSTEM REQUIREMENTS 3.1. Hardware Requirement 3.2. Software Requirement 3.3. Hardware Description 3.4. Software description	**7**
4	PROJECT DESRIPTION 4.1. Name Of The Modules 4.2. Module Description	**20**
5	SYSTEM DESIGN 5.1. Dataflow Diagram 5.2. Database Design	**23**
6	IMPLEMENTATION	**26**
7	TESTING AND MAINTENANCE 7.1. Functional Testing 7.2. Performance Testing 7.3. Security Testing	**27**
8	APPENDICES Appendix-1 Appendix-2	**29**
9	CONCLUSION & FUTURE	**52**
10	BIBLIOGRAPHY	**54**

ABSTRACT

This project introduces a **real-time, bidirectional communication system** designed to bridge interactions between hearing individuals and people who are deaf or speech-impaired. The system translates **sign language into speech** and converts **spoken language into animated visual gestures**, enabling natural and inclusive communication. Using **computer vision** and **deep learning**, the system recognizes both **static and dynamic hand gestures** through live video streams. These gestures are transformed into **textual representation** and **speech synthesis**, facilitating comprehension for non-signers. Conversely, **automatic speech recognition** processes spoken input and generates corresponding sign language animations via a **virtual avatar**, ensuring accurate and intuitive feedback.The architecture emphasizes **low latency**, **high accuracy**, and **real-time performance**, making it suitable for educational, professional, and public service environments. It supports **multilingual adaptability**, allowing integration with various spoken languages and sign language dialects. By combining **gesture recognition**, **speech processing**, and **avatar-based visualization**, the system promotes **social inclusion**, enhances **human-computer interaction**, and reduces communication barriers.This work demonstrates the practical application of **artificial intelligence** in **assistive technologies**, fostering equitable participation for individuals with hearing and speech impairments. The framework's **scalable and user-friendly design** ensures accessibility across diverse contexts, illustrating the potential of **multimodal AI systems** to improve quality of life. By uniting advanced machine learning techniques with **interactive visualization**, the system offers a reliable and efficient solution for seamless cross-modal communication, highlighting a path toward inclusive digital interaction.

Keywords:

Sign Language Translation, Deep Learning, Computer Vision, Speech Recognition, Virtual Avatars, Accessibility, Assistive Technology, Real-Time Communication, Gesture Recognition, Speech Synthesis, Bidirectional Interaction

CHAPTER 1

1. INTRODUCTION

Communication is a fundamental human need, yet millions of people with hearing or speech impairments continue to face barriers in education, healthcare, employment, and social interaction. Sign language serves as a primary means of communication for the deaf community, but most hearing individuals are not trained to understand it, resulting in limited inclusion and reduced participation in daily activities. Traditional communication aids such as interpreters and text-based tools are often unavailable, costly, or impractical in real-time situations, further widening the communication gap.

Recent advancements in artificial intelligence, computer vision, and speech processing have created new opportunities to design intelligent systems that can understand human gestures and spoken language. Deep learning models, particularly Convolutional Neural Networks, have demonstrated remarkable success in recognizing complex visual patterns, including static and dynamic hand gestures used in sign language. Similarly, automatic speech recognition and natural language processing technologies have significantly improved the accuracy and speed of speech-to-text conversion.

By integrating these technologies with virtual avatar systems, it becomes possible to visually represent spoken language as animated sign gestures and convert sign language into natural speech. This approach enables bidirectional communication, allowing both hearing and non-hearing individuals to interact naturally without requiring prior knowledge of sign language. The use of avatars further enhances user engagement by providing clear, expressive, and consistent sign representations.

This project introduces a real-time translation system that combines gesture recognition, speech processing, and avatar-based visualization to support inclusive communication across diverse environments. The system is designed to operate efficiently in real-world settings such as classrooms, hospitals, workplaces, and public spaces. By leveraging deep learning and interactive technologies, the proposed solution aims to reduce communication barriers, promote social inclusion, and improve the quality of life for individuals with hearing and speech impairments.

1.1 OVERVIEW OF THE PROJECT

This project focuses on developing a real-time bidirectional translation system that enables seamless communication between hearing individuals and those who are deaf or speech-impaired. By integrating deep learning, computer vision, speech processing, and virtual avatar technologies, the system bridges the communication gap between sign language users and non-signers. The solution captures live video input to recognize hand gestures using Convolutional Neural Networks and converts the recognized signs into readable text and natural speech. In the reverse process, spoken language is transformed into text through automatic speech recognition and displayed as animated sign gestures using a virtual avatar.

The system is designed to operate efficiently in real-world environments such as educational institutions, healthcare facilities, workplaces, and public service centers. Its user-friendly interface ensures ease of use for both sign language users and hearing individuals, promoting inclusive interaction without the need for human interpreters. The project emphasizes accuracy, real-time performance, and adaptability to multiple sign and spoken languages, making it suitable for diverse communication contexts. By combining advanced machine learning techniques with intuitive visualization, the proposed system enhances accessibility, supports social inclusion, and contributes to the development of intelligent assistive technologies that improve the quality of life for individuals with communication disabilities worldwide.

1.2 Problem Description

Communication barriers between hearing individuals and people with hearing or speech impairments remain a significant social and technological challenge. Although sign language serves as an effective communication medium for the deaf community, most hearing individuals are not familiar with it, creating a persistent gap in understanding and interaction. This lack of mutual communication limits access to essential services, education, employment opportunities, and social participation, often leading to isolation and reduced quality of life.

Existing communication support systems, such as human interpreters, text messaging, and captioning tools, offer partial solutions but suffer from several limitations. Human interpreters are not always available, can be expensive, and may not be practical in emergency or spontaneous situations. Text-based communication lacks the expressiveness of sign language and is unsuitable for individuals with limited literacy skills. Furthermore, many current automated sign language recognition systems are limited in vocabulary, accuracy, and real-time performance, restricting their effectiveness in real-world applications.

On the other hand, speech recognition systems designed for general use often fail to address the specific needs of communication with deaf or speech-impaired individuals, particularly in noisy environments or multilingual settings. Additionally, most existing solutions operate in a single direction, either translating speech to text or recognizing gestures, but do not support complete bidirectional communication.

The absence of a reliable, real-time, and user-friendly system capable of translating both sign language into speech and speech into visual signs creates a major accessibility gap. This gap prevents inclusive interaction between sign language users and non-signers, especially in critical domains such as healthcare, education, customer service, and public safety.

Therefore, there is a pressing need for an intelligent, scalable, and accurate communication system that integrates gesture recognition, speech processing, and visual representation technologies. Such a system must support natural interaction, operate in real-time, and adapt to diverse linguistic contexts, ensuring equal access to communication for individuals with hearing and speech impairments across society.

CHAPETR 2

2. SYSTEM ANALYSIS

2.1. EXISTING SYSTEM

Existing communication systems for individuals with hearing or speech impairments rely primarily on human interpreters, text-based tools, and limited automated technologies. Human sign language interpreters provide accurate and expressive translation, but their availability is often restricted, especially in rural areas, emergency situations, or spontaneous interactions. The cost of professional interpretation services can also be high, making them inaccessible for many individuals and organizations. Additionally, interpreter-dependent communication may raise privacy concerns in sensitive environments such as healthcare consultations or legal discussions.

Text-based communication tools, including mobile messaging, captioning services, and speech-to-text applications, offer partial support for interaction. While these tools are useful for basic communication, they lack the visual and expressive richness of sign language, which is the primary language for many deaf individuals. Text-based methods also require literacy skills and may not be effective for users with limited reading or writing proficiency. Furthermore, such systems do not accommodate the grammatical structure and cultural nuances of sign languages, reducing communication accuracy and naturalness.

Some automated sign language recognition systems have been developed using image processing and machine learning techniques. However, many of these systems are limited to recognizing a small set of static gestures and often struggle with dynamic signs, facial expressions, and contextual meaning. Their performance is frequently affected by variations in lighting, background, camera quality, and user posture, leading to reduced reliability in real-world settings. Additionally, most of these systems operate in a unidirectional manner, translating sign language into text or speech without supporting reverse translation.

Speech recognition technologies have advanced significantly, yet they are primarily designed for general speech input and may not perform effectively in noisy environments or when interacting with individuals who have speech impairments. These systems also fail to provide visual sign language output, limiting their usefulness for deaf users.

Disadvantages of the Existing System

- ✓ **Limited Availability of Interpreters:** Human interpreters are not always accessible, especially in emergencies or remote locations.
- ✓ **High Cost:** Professional interpretation services can be expensive, making them unaffordable for many users.
- ✓ **Lack of Real-Time Automation:** Most systems do not support instant, automated bidirectional communication.
- ✓ **Low Accuracy in Automated Systems:** Existing gesture recognition tools struggle with dynamic signs and complex expressions.
- ✓ **Poor User Experience:** Many solutions are not user-friendly and fail to support natural, inclusive interaction.

2.2. PROPOSED SYSTEM

The proposed system introduces a real-time bidirectional communication platform that enables seamless interaction between hearing individuals and those who are deaf or speech-impaired. It integrates deep learning, computer vision, speech recognition, and virtual avatar technologies to provide an accurate, efficient, and user-friendly translation mechanism between sign language and spoken language. The system captures live video input through a camera to recognize hand gestures using Convolutional Neural Networks trained on large sign language datasets. These recognized gestures are converted into meaningful text and synthesized speech, allowing sign language users to communicate naturally with non-signers.

In the reverse direction, spoken input is processed using automatic speech recognition to convert speech into text. This text is then mapped to corresponding sign language gestures and rendered visually using an animated virtual avatar. The avatar performs expressive and accurate signs, enabling deaf users to understand spoken messages clearly and intuitively. This bidirectional translation ensures inclusive communication without the need for human interpreters.

The system is designed for real-time operation, offering low latency and high accuracy even in dynamic environments. It is adaptable to multiple sign languages and spoken languages, making it suitable for diverse linguistic and cultural contexts. A user-friendly interface ensures ease of use for individuals of all technical backgrounds, promoting widespread adoption.

The proposed solution emphasizes scalability, reliability, and accessibility, making it suitable for deployment in educational institutions, healthcare facilities, workplaces, customer service centers, and public spaces. By combining advanced machine learning models with interactive visualization techniques, the system enhances communication efficiency and social inclusion. Ultimately, this project aims to bridge the communication gap between sign language users and non-signers, improve the quality of life for individuals with hearing and speech impairments, and contribute to the development of intelligent assistive technologies that support equal access to communication worldwide.

ADVANTAGES OF THE PROPOSED SYSTEM

- ✓ **Real-Time Communication:** Enables instant translation between sign language and speech without delays.
- ✓ **Bidirectional Translation:** Supports both sign-to-speech and speech-to-sign interaction for complete communication.
- ✓ **High Accuracy:** Uses deep learning models to recognize gestures and speech reliably.
- ✓ **User-Friendly Interface:** Designed for easy use by both deaf and hearing individuals.
- ✓ **Improved Accessibility:** Promotes inclusion in education, healthcare, workplaces, and public services.

CHAPTER 3

3. SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENT:-

- PROCESS: INTEL® CORE™ I9-14900K 3.20 GHZ
- RAM: 16 GB
- HARD DISK: 1 TB

3.2 SOFTWARE REQUIREMENT:-

- FRONTEND : HTML, CSS
- BACKEND : PYTHON
- FRAMEWORK : FLASK

3.3 HARDWARE DECRIPTION

PROCESSOR : INTEL® CORE™ I9-14900K 3.20 GHZ

The **Intel® Core™ i9-14900K processor** with a base clock speed of 3.20 GHz is a powerhouse in terms of computational capabilities. Designed for intensive workloads, it offers exceptional multi-core performance, making it ideal for tasks like gaming, video editing, 3D rendering, and other processor-intensive applications. Its high clock speed and modern architecture ensure efficient data processing, quick task execution, and minimal latency.

2. RAM

With **16 GB of RAM**, the system provides ample memory for seamless multitasking. This capacity supports the smooth operation of modern software, gaming, and professional applications like video editing tools or machine learning frameworks. It ensures that switching between applications remains fluid, without system slowdowns.

3. HARD DISK

The **1 TB hard disk** offers a significant amount of storage space to accommodate the operating system, essential software, media files, and other data. While traditional hard drives provide large storage capacities, combining this with an SSD can greatly enhance system performance, particularly in boot times and file transfers.

3.4 SOFTWARE DECRIPTION

FRONTEND

INTRODUCTION TO HTML

HTML, which stands for Hypertext Markup Language, is the standard markup language for creating web pages. It provides the structure for web documents by using a system of tags and attributes to define elements within the page. These elements can include headings, paragraphs, images, links, forms, and more.

Working Process

HTML documents are text files that contain a series of elements enclosed in angle brackets (<>). These elements are organized in a hierarchical structure, with the <html> element at the top, followed by <head> and <body> elements. The <head> section typically contains meta-information about the document, such as its title and links to external resources like stylesheets and scripts. The <body> section contains the content visible to the user.

Within the <body> section, elements like <p> for paragraphs, <h1> to <h6> for headings, <img> for images, and <a> for links are used to create the desired layout and functionality of the webpage. Attributes can be added to these elements to provide additional information or modify their behavior. Once an HTML document is created, it can be viewed in a web browser, which interprets the HTML code and displays the content according to the specified structure and formatting. Additionally, HTML can be enhanced with the use of CSS (Cascading Style Sheets) for styling and JavaScript for interactivity, allowing for more dynamic and visually appealing web pages.

CASCADING STYLE SHEETS

INTRODUCTION TO CSS

CSS, short for Cascading Style Sheets, is a style sheet language used to describe the presentation of a document written in HTML or XML. It controls the layout, formatting, and appearance of web pages, allowing developers to define the visual aspects such as colors, fonts, spacing, and positioning**.**

Working Process

CSS works by targeting HTML elements and applying styling rules to them. These rules consist of selectors that identify which elements to style and declarations that specify the styling properties and values. Selectors can target elements based on their tag names, classes, IDs, attributes, or even their relationship with other elements in the document. Once selected, CSS properties such as color, font-size, margin, padding, and border can be applied to change the appearance of the elements.

CSS can be applied to HTML documents in three ways: inline styles, internal styles, and external stylesheets. Inline styles are applied directly within the HTML tags using the "style" attribute, internal styles are defined within the <style> element in the head section of the HTML document, and external stylesheets are separate CSS files linked to the HTML document using the <link> element. When a web browser renders an HTML document, it interprets the CSS rules and applies the specified styles to the corresponding elements, resulting in the desired visual presentation of the webpage.

BACKEND PYTHON

Python Technology:

Python is an interpreter, high-level, general-purpose programming language. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. **Python** is often described as a "batteries included" language due to its comprehensive standard library.

Python Programing Language:

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of its features support functional programming and aspect-oriented programming (including by Meta programming and met objects (magic methods)). Many other paradigms are supported via extensions, including design by contract and logic programming.

Python packages with a wide range of functionality, including:

- Easy to Learn and Use
- Expressive Language
- Interpreted Language
- Cross-platform Language
- Free and Open Source
- Object-Oriented Language
- Extensible
- Large Standard Library
- GUI Programming Support
- Integrated

Python uses dynamic typing and a combination of reference counting and a cycle-detecting garbage collector for memory management. It also features dynamic name resolution (late binding), which binds method and variable names during program execution.

Rather than having all of its functionality built into its core, Python was designed to be highly extensible. This compact modularity has made it particularly popular as a means of adding programmable interfaces to existing applications. Van Rossum's vision of a small core language with a large standard library and easily extensible interpreter stemmed from his frustrations with ABC, which espoused the opposite approach.

Python strives for a simpler, less-cluttered syntax and grammar while giving developers a choice in their coding methodology. In contrast to Perl's "there is more than one way to do it" motto, Python embraces a "there should be one and preferably only one obvious way to do it" design philosophy. Alex Martelli, a Fellow at the Python Software Foundation and Python book author, writes that "To describe something as 'clever' is not considered a compliment in the Python culture."

Python's developers strive to avoid premature optimization, and reject patches to non-critical parts of the Python reference implementation that would offer marginal increases in speed at the cost of clarity. When speed is important, a Python programmer can move time-critical functions to extension modules written in languages such as C, or use PyPy, a just-in-time compiler. Python is also available, which translates a Python script into C and makes direct C-level API calls into the Python interpreter.

What does python technology do?

Python is quite popular among programmers, but the practice shows that business owners are also Python development believers and for good reason. Software developers love it for its straightforward syntax and reputation as one of the easiest programming languages to learn. Business owners or CTOs appreciate the fact that there’s a framework for pretty much anything – from web apps to machine learning.

Moreover, it is not just a language but more a technology platform that has come together through a gigantic collaboration from thousands of individual professional developers forming a huge and peculiar community of aficionados.

PRODUCTIVITY AND SPEED

It is a widespread theory within development circles that developing Python applications is approximately up to 10 times faster than developing the same application in Java or C/C++. The impressive benefit in terms of time saving can be explained by the clean object-oriented design, enhanced process control capabilities, and strong integration and text processing capacities. Moreover, its own unit testing framework contributes substantially to its speed and productivity.

PYTHON IS POPULAR FOR WEB APPS

Web development shows no signs of slowing down, so technologies for rapid and productive web development still prevail within the market. Along with JavaScript and Ruby, Python, with its most popular web framework Django, has great support for building web apps and is rather popular within the web development community.

OPEN-SOURCE AND FRIENDLY COMMUNITY

As stated on the official website, it is developed under an OSI-approved open source license, making it freely usable and distributable. Additionally, the development is driven by the community, actively participating and organizing conference, meet-ups, hackathons, etc. fostering friendliness and knowledge-sharing.

PYTHON IS QUICK TO LEARN

It is said that the language is relatively simple so you can get pretty quick results without actually wasting too much time on constant improvements and digging into the complex engineering insights of the technology. Even though Python programmers are really in high demand these days, its friendliness and attractiveness only help to increase number of those eager to master this programming language.

BROAD APPLICATION

It is used for the broadest spectrum of activities and applications for nearly all possible industries. It ranges from simple automation tasks to gaming, web development, and even complex enterprise systems. These are the areas where this technology is still the king with no or little competence:

- Machine learning as it has a plethora of libraries implementing machine learning algorithms.
- Web development as it provides back end for a website or an app.
- Cloud computing as Python is also known to be among one of the most popular cloud-enabled languages even used by Google in numerous enterprise-level software apps.
- Scripting.
- Desktop GUI applications.

Python compiler

The Python compiler package is a tool for analyzing Python source code and generating Python bytecode. The compiler contains libraries to generate an abstract syntax tree from Python source code and to generate Python bytecode from the tree.

The compiler package is a Python source to bytecode translator written in Python. It uses the built-in parser and standard parser module to generate a concrete syntax tree. This tree is used to generate an abstract syntax tree (AST) and then Python bytecode.

The full functionality of the package duplicates the built-in compiler provided with the Python interpreter. It is intended to match its behavior almost exactly. Why implement another compiler that does the same thing? The package is useful for a variety of purposes. It can be modified more easily than the built-in compiler. The AST it generates is useful for analyzing Python source code.

The basic interface

The top-level of the package defines four functions. If you import compiler, you will get these functions and a collection of modules contained in the package.

compiler.parse(buf)

Returns an abstract syntax tree for the Python source code in buf. The function raises Syntax Error if there is an error in the source code. The return value is a compiler.ast. Module instance that contains the tree.

compiler.parseFile(path)

Return an abstract syntax tree for the Python source code in the file specified by path. It is equivalent to parse(open(path).read()).

LIMITATIONS

There are some problems with the error checking of the compiler package. The interpreter detects syntax errors in two distinct phases. One set of errors is detected by the interpreter's parser, the other set by the compiler. The compiler package relies on the interpreter's parser, so it get the first phases of error checking for free. It implements the second phase itself, and that implementation is incomplete. For example, the compiler package does not raise an error if a name appears more than once in an argument list: deff(x, x): ...

A future version of the compiler should fix these problems.

All Node objects offer the following methods:

getChildren()

Returns a flattened list of the child nodes and objects in the order they occur. Specifically, the order of the nodes is the order in which they appear in the Python grammar. Not all of the children are Node instances. The names of functions and classes, for example, are plain strings.

getChildNodes()

Returns a flattened list of the child nodes in the order they occur. This method is like getChildren(), except that it only returns those children that are Node instances.

The While node has three attributes: test, body, and else_. (If the natural name for an attribute is also a Python reserved word, it can't be used as an attribute name. An underscore is appended to the word to make it a legal identifier, hence else_ instead of else.)

The if statement is more complicated because it can include several tests.

The If node only defines two attributes: tests and else_. The tests attribute is a sequence of test expression, consequent body pairs. There is one pair for each if/elif clause. The first element of the pair is the test expression. The second elements is a Stmt node that contains the code to execute if the test is true.

The getChildren() method of If returns a flat list of child nodes. If there are three if/elif clauses and no else clause, then getChildren() will return a list of six elements: the first test expression, the first Stmt, the second text expression, etc.

DEVELOPMENT ENVIRONMENTS:

Most Python implementations (including CPython) include a read–eval–print loop (REPL), permitting them to function as a command line interpreter for which the user enters statements sequentially and receives results immediately.

Other shells, including IDLE and IPython, add further abilities such as auto-completion, session state retention and syntax highlighting.

IMPLEMENTATIONS

Reference implementation

CPython is the reference implementation of Python. It is written in C, meeting the C89 standard with several select C99 features. It compiles Python programs into an intermediate bytecode which is then executed by its virtual machine. CPython is distributed with a large standard library written in a mixture of C and native Python. It is available for many platforms,

including Windows and most modern Unix-like systems. Platform portability was one of its earliest priorities.

Unsupported implementations

Other just-in-time Python compilers have been developed, but are now unsupported:

Google began a project named Unladen Swallow in 2009, with the aim of speeding up the Python interpreter five-fold by using the LLVM, and of improving its multithreading ability to scale to thousands of cores, while ordinary implementations suffer from the global interpreter lock.

Psyco is a just-in-time specializing compiler that integrates with CPython and transforms bytecode to machine code at runtime. The emitted code is specialized for certain data types and is faster than standard Python code.

Cross-compilers to other languages

There are several compilers to high-level object languages, with either unrestricted Python, a restricted subset of Python, or a language similar to Python as the source language:

- Jython enables the use of the Java class library from a Python program.
- IronPython follows a similar approach in order to run Python programs on the .NET Common Language Runtime.
- The RPython language can be compiled to C, and is used to build the PyPy interpreter of Python.
- Pyjs compiles Python to JavaScript.
- Cython compiles Python to C and C++.
- Numba uses LLVM to compile Python to machine code.
- Pythran compiles Python to C++.
- Somewhat dated Pyrex (latest release in 2010) and Shed Skin (latest release in 2013) compile to C and C++ respectively.
- Google's Grumpy compiles Python to Go.
- MyHDL compiles Python to VHDL.
- Nuitka compiles Python into C++.

PERFORMANCE

A performance comparison of various Python implementations on a non-numerical (combinatorial) workload was presented at EuroSciPy '13.

API DOCUMENTATION GENERATORS

Python API documentation generators include:

- Sphinx
- Epydoc
- HeaderDoc
- Pydoc

USES

Python has been successfully embedded in many software products as a scripting language, including in finite element method software such as Abaqus, 3D parametric modeler like FreeCAD, 3D animation packages such as 3ds Max, Blender, Cinema 4D, Lightwave, Houdini, Maya, modo, MotionBuilder, Softimage, the visual effects compositor Nuke, 2D imaging programs like GIMP, Inkscape, Scribus and Paint Shop Pro, and musical notation programs like scorewriter and capella. GNU Debugger uses Python as a pretty printer to show complex structures such as C++ containers. Esri promotes Python as the best choice for writing scripts in ArcGIS. It has also been used in several video games, and has been adopted as first of the three available programming languages in Google App Engine, the other two being Java and Go.

Python is commonly used in artificial intelligence projects with the help of libraries like TensorFlow, Keras and Scikit-learn. As a scripting language with modular architecture, simple syntax and rich text processing tools, Python is often used for natural language processing.

Many operating systems include Python as a standard component. It ships with most Linux distributions, AmigaOS 4, FreeBSD (as a package), NetBSD, OpenBSD (as a package) and macOS and can be used from the command line (terminal). Many Linux distributions use installers written in Python: Ubuntu uses the Ubiquity installer, while Red Hat Linux and Fedora

use the Anaconda installer. Gentoo Linux uses Python in its package management system, Portage.

Python is used extensively in the information security industry, including in exploit development.

Most of the Sugar software for the One Laptop per Child XO, now developed at Sugar Labs, is written in Python. The Raspberry Pi single-board computer project has adopted Python as its main user-programming language.

LibreOffice includes Python, and intends to replace Java with Python. Its Python Scripting Provider is a core feature since Version 4.0 from 7 February 2013.

PANDAS

In computer programming, pandas is a software library written for the Python programming language for data manipulation and analysis. In particular, it offers data structures and operations for manipulating numerical tables and time series. It is free software released under the three-clause BSD license. The name is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods for the same individuals.

Library features

- Data Frame object for data manipulation with integrated indexing.
- Tools for reading and writing data between in-memory data structures and different file formats.
- Data alignment and integrated handling of missing data.
- Reshaping and pivoting of data sets.
- Label-based slicing, fancy indexing, and sub setting of large data sets.
- Data structure column insertion and deletion.
- Group by engine allowing split-apply-combine operations on data sets.
- Data set merging and joining.
- Hierarchical axis indexing to work with high-dimensional data in a lower-dimensional data structure.

- Time series-functionality: Date range generation and frequency conversion, moving window statistics, moving window linear regressions, date shifting and lagging.

FLASK

Flask is a lightweight and flexible web framework written in Python, widely used for developing web applications and APIs. Unlike heavier frameworks such as Django, Flask follows a micro-framework philosophy, meaning it provides the essentials for web development but allows developers to choose additional tools and libraries based on project requirements. This makes Flask very versatile and an excellent choice for small to medium-sized projects, prototypes, or microservices.

At its core, Flask provides tools for routing, request handling, and template rendering, making it easy to create robust applications with minimal boilerplate code. It uses Werkzeug as its WSGI (Web Server Gateway Interface) toolkit and Jinja2 as its templating engine, allowing developers to dynamically render HTML pages and build clean, organized projects. Because Flask is unopinionated, developers have complete freedom over project structure, database selection, and additional features such as authentication or caching.

Flask is highly extensible through its rich ecosystem of extensions. Popular libraries like Flask-SQLAlchemy (for database integration), Flask-Login (for authentication), and Flask-RESTful (for building REST APIs) allow developers to add functionality easily. This modularity is one of Flask's biggest advantages, giving developers full control over which features to include and keeping applications lightweight.

Another benefit of Flask is its simplicity and readability, which makes it beginner-friendly. The framework is easy to learn and quick to set up, making it ideal for hackathons, proofs of concept, and early-stage startups. At the same time, it is powerful enough to be used in production environments by large organizations such as Netflix, Lyft, and Reddit, which rely on Flask for building scalable services.

Flask also works well with modern front-end frameworks such as React or Angular, allowing developers to build full-stack applications where Flask handles the backend APIs.

CHAPTER 4

4. PROJECT DESRIPTION

4.1. NAME OF THE MODULES

1. User Interface Module
2. Gesture Recognition Module
3. Speech Recognition Module
4. Text Processing and Mapping Module
5. Speech Synthesis Module
6. Virtual Avatar Module
7. Database Management Module

4.2. MODULE DESCRIPTION

1. User Interface Module

The User Interface Module serves as the primary interaction point between the system and end users, accommodating both sign language users and hearing individuals. It enables users to input gestures via a camera or speech through a microphone while providing clear output in the form of text, speech, or animated avatars. The interface is designed for intuitiveness, accessibility, and responsiveness, ensuring ease of use across diverse users. It manages real-time input capture, visual feedback, and output display, while coordinating with other system modules. Its design prioritizes inclusivity, simplicity, and adaptability to different environments and user needs.

2. Gesture Recognition Module

The Gesture Recognition Module interprets static and dynamic hand gestures captured through the camera. Using Convolutional Neural Networks and image processing algorithms, it identifies intricate patterns, finger positions, and movement sequences to convert gestures into accurate textual representations. The module handles variations in lighting, background, and user posture while maintaining real-time performance. It interacts with the Text Processing Module to ensure recognized gestures correspond to correct semantic meaning. By providing high accuracy and low latency recognition, this module forms the foundation for translating sign language into speech or text, supporting inclusive communication for individuals with hearing impairments.

3. Speech Recognition Module

The Speech Recognition Module captures audio input from users and converts spoken language into corresponding text. It employs advanced automatic speech recognition algorithms and deep learning models to accurately process different accents, speech patterns, and environmental noise conditions. The module integrates with the Text Processing and Mapping Module to translate recognized speech into sign language sequences displayed through the virtual avatar. Designed for real-time responsiveness, it ensures minimal latency to maintain natural communication flow. By bridging spoken language input to visual sign representation, this module allows hearing users to communicate effectively with sign language users, promoting inclusive and bidirectional interaction.

4. Text Processing and Mapping Module

The Text Processing and Mapping Module acts as the system's central translator, converting recognized gestures and speech-derived text into corresponding outputs. It maps gesture patterns to precise textual or speech representations and converts speech-recognized text into appropriate sign sequences for the virtual avatar. This module ensures semantic accuracy, resolves ambiguities, and maintains contextual meaning across different languages and sign conventions. It coordinates with the Database Management Module to access updated gesture and language mappings.

5. Speech Synthesis Module

The Speech Synthesis Module converts processed text, derived from recognized gestures, into natural, intelligible speech for hearing users. Using text-to-speech algorithms and voice modeling techniques, it produces clear, human-like audio output that accurately represents the meaning of the original sign language input. The module ensures real-time performance, minimizing delays to maintain fluid conversation. It integrates with the Text Processing Module to receive accurate textual content and supports multiple languages and voice settings. By enabling sign language users to communicate vocally with non-signers, this module enhances inclusivity, accessibility, and bidirectional communication, facilitating natural interaction in educational, professional, and public environments.

6. Virtual Avatar Module

The Virtual Avatar Module visually represents recognized speech as sign language gestures through an animated, expressive avatar. It receives processed text from the Text Processing and Mapping Module and renders corresponding sign sequences with correct hand positions, movements, and facial expressions. This module ensures clarity, consistency, and naturalness in sign language visualization, supporting real-time translation. By providing a lifelike representation, the avatar enables deaf or speech-impaired users to understand spoken language intuitively. It works closely with the Database Module to access gesture animations and linguistic rules. This module enhances engagement, comprehension, and accessibility, promoting inclusive communication across diverse environments.

7. Database Management Module

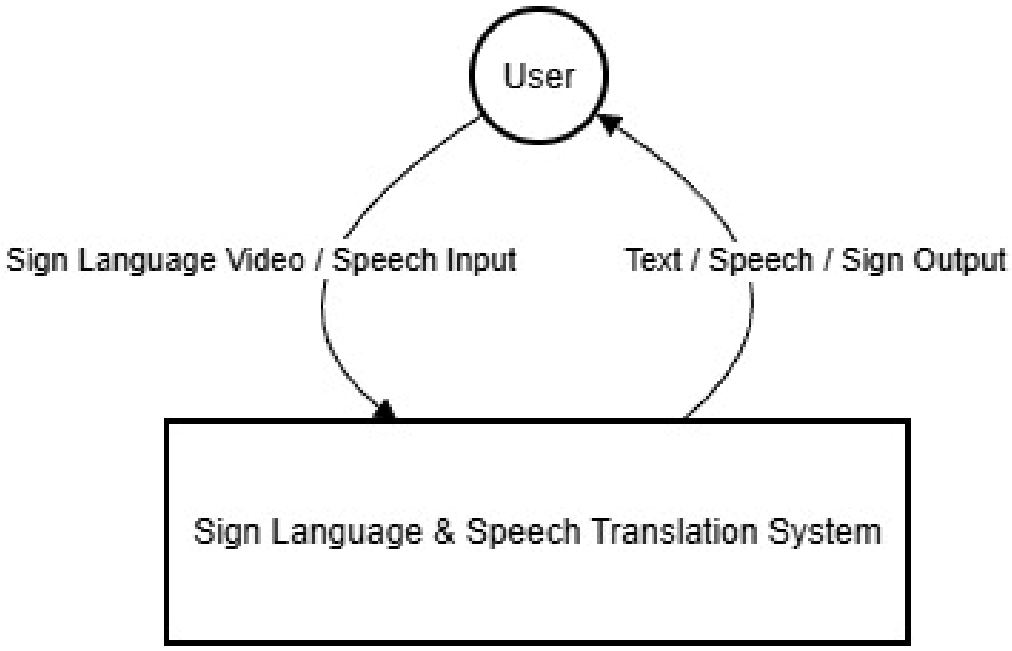
The Database Management Module serves as the centralized repository for all gesture data, speech mappings, avatar animations, and user interaction logs. It stores static reference datasets, dynamic session records, and configurable mappings to ensure accurate, efficient, and scalable translation. The module supports quick retrieval of gesture-to-text, speech-to-sign, and avatar rendering information, enabling real-time performance. It also maintains system integrity, consistency, and security through indexing, normalization, and backup mechanisms. By coordinating with all other modules, it ensures seamless data flow, facilitates learning-based model updates, and enables

CHAPTER 5

5. SYSTEM DESIGN

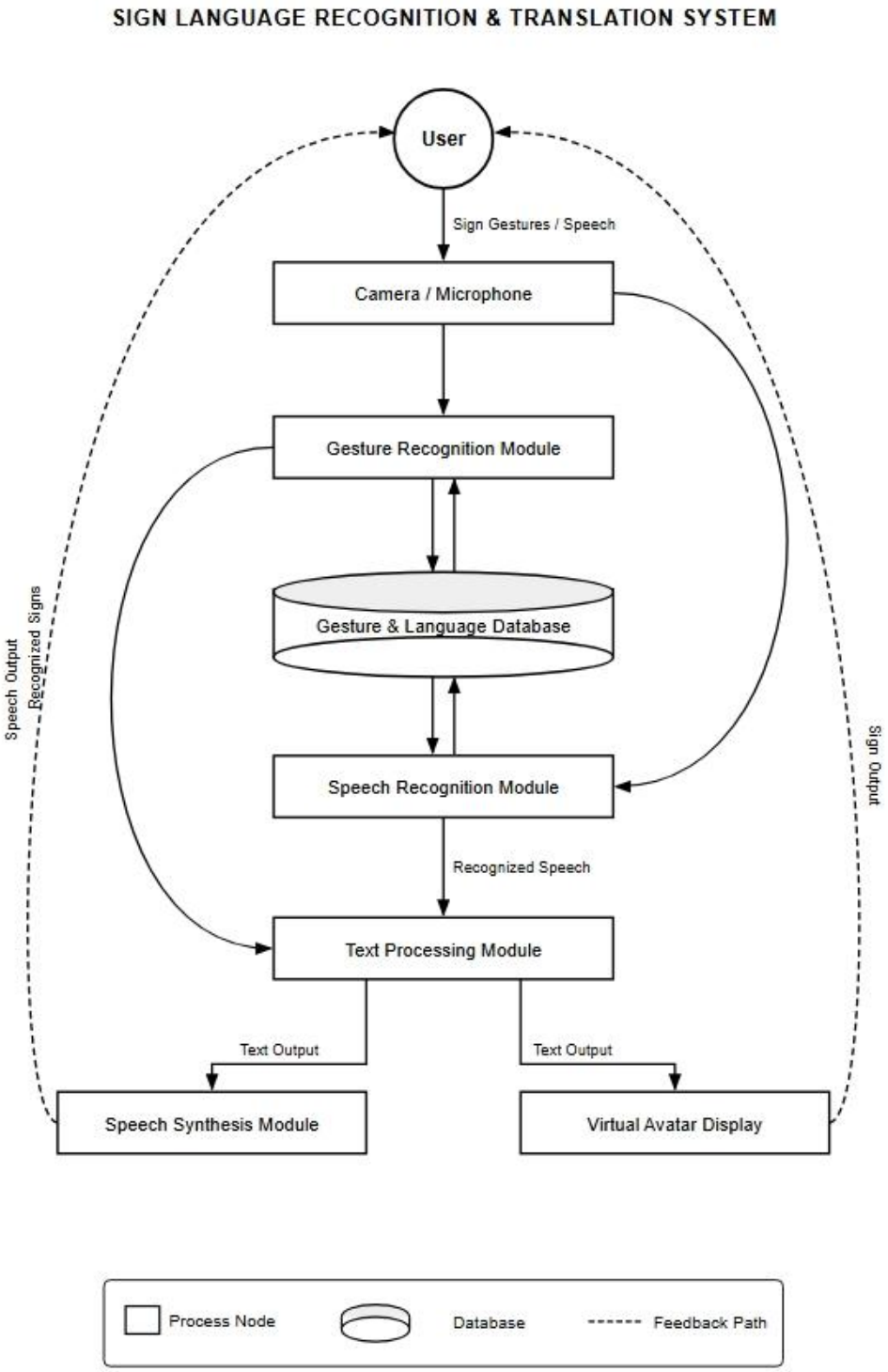
5.1. DATAFLOW DIAGRAM

LEVEL 0



ISL CONVERTER 5.1.1

LEVEL 1



SYSTEM DESIGN 5.1.2

5.2. DATABASE DESIGN

The database design for the **Real-Time Sign Language and Speech Translation System** is structured to efficiently store, manage, and retrieve data required for accurate gesture recognition, speech processing, and avatar-based visualization. The system employs a relational database to maintain consistency, integrity, and fast access for real-time operations. The key components of the database include gesture datasets, sign language mappings, speech-to-text records, avatar animation data, and user interaction logs.

Key Tables:

1. **Users Table:** Stores user information such as UserID, Name, Contact, and User Type (Deaf, Hearing, Admin) to manage personalized interactions and preferences.
2. **Gestures Table:** Maintains details of static and dynamic gestures with fields like GestureID, Gesture Name, Image/Video Reference, and Gesture Category.
3. **Sign Language Mappings Table:** Links recognized gestures to corresponding text or speech outputs, enabling accurate translation between visual and verbal languages.
4. **Speech Records Table:** Stores speech input, recognized text, and timestamps to assist in processing, analytics, and training improvements.
5. **Avatar Actions Table:** Contains animation sequences, gestures, and rendering parameters for the virtual avatar to display correct signs based on text input.
6. **Logs Table:** Captures real-time system interactions, including user inputs, outputs generated, errors, and system performance metrics for monitoring and future enhancements.

The database ensures smooth bidirectional translation by providing a structured and optimized repository for both static reference data (gestures, mappings, avatar animations) and dynamic interaction data (speech input, recognized text, user sessions). It supports scalability for additional sign languages, new gestures, or speech patterns, making the system adaptable for diverse linguistic and cultural contexts. Proper indexing, normalization, and backup mechanisms ensure

CHAPTER 6

6. IMPLEMENTATION

The system is implemented as a **real-time bidirectional framework** integrating **computer vision, deep learning, speech recognition, and virtual avatar technologies**. It consists of two primary modules: **Sign-to-Speech (STS)** and **Speech-to-Sign (S2S)**.

In the STS module, live video input is processed to detect **hand gestures**, including both static and dynamic signs. **Convolutional Neural Networks (CNNs)** are employed for accurate gesture classification, while **hand tracking algorithms** identify spatial and temporal features. Recognized gestures are converted into **text**, which is then transformed into **speech** using **Text-to-Speech (TTS)** engines. This enables sign language users to communicate naturally with hearing individuals.

The S2S module captures audio input and processes it using **Automatic Speech Recognition (ASR)** to convert speech into text. The text is mapped to corresponding sign language gestures through a **gesture dictionary** or sequence models. A **virtual avatar** performs these gestures using **3D animation**, providing a realistic and intuitive visual representation.

The two modules are integrated through a **real-time pipeline**, ensuring **low-latency** and synchronized translation. GPU acceleration is utilized to optimize deep learning inference, maintaining smooth performance during continuous interaction. The system is designed for **scalability**, supporting multiple sign languages and spoken languages with minimal adaptation.

A **user-friendly interface** displays live video, recognized text, and animated gestures, allowing users to select translation direction, language, and feedback type. This implementation demonstrates the practical application of **AI and multimodal systems** in **assistive technology**, enhancing **accessibility**, **inclusion**, and **human-computer interaction** for individuals with hearing and speech impairments.

CHAPTER 7

7. TESTING AND MAINTENANCE

Testing ensures that the **Self-Healing Cloud Infrastructure Using AI** performs reliably, efficiently, and securely. It validates the system's capability to monitor, detect, predict, and recover from failures while maintaining high availability. Three major testing types are implemented:

7.1. Functional Testing

Functional testing verifies that each module of the self-healing cloud system operates according to its specifications. The **Monitoring Module** is tested to ensure accurate real-time metric collection from servers, networks, and applications. The **Data Collection & Storage Module** is validated for proper aggregation, storage, and retrieval of large-scale data sets. Anomaly detection algorithms are assessed to confirm they correctly identify deviations from normal behavior without false positives. Predictive analysis models are evaluated for their accuracy in forecasting potential failures, while the automated recovery module is tested to ensure dynamic scaling, service migration, and configuration adjustments occur correctly. Reporting and logging functionalities are verified to provide timely alerts and comprehensive system reports. Functional testing also ensures seamless communication between modules, confirming the closed-loop self-healing workflow operates as designed under normal and stressed conditions.

7.2. Performance Testing

Performance testing assesses the system's responsiveness, scalability, and resource efficiency under varying workloads. The monitoring and data collection modules are evaluated for their ability to handle high-frequency metrics from multiple virtual machines and containers simultaneously. Predictive models are tested for real-time processing speed, ensuring anomalies and potential failures are detected quickly. Automated recovery actions, such as resource scaling or service migration, are timed to measure downtime minimization and system resilience. Automated recovery actions are tested to ensure they do not inadvertently expose vulnerabilities while reallocating resources or migrating services. Regular security audits simulate breach attempts and measure the system's capacity to maintain integrity and continuity. The system's

performance under peak loads, network congestion, or sudden spikes in application requests is also examined. Resource utilization efficiency is measured, confirming optimal allocation of CPU, memory, and bandwidth without over-provisioning. Performance testing ensures the AI-driven cloud system maintains stability, high throughput, and low latency while executing predictive and corrective actions. This testing guarantees that the infrastructure can scale dynamically while sustaining reliable service delivery.

7.3. Security Testing

Security testing evaluates the robustness of the self-healing cloud infrastructure against potential threats and unauthorized access. The system's monitoring and logging modules are tested to ensure sensitive data is encrypted during storage and transmission. Access control mechanisms are verified to prevent unauthorized modification of AI models or automated recovery scripts. The anomaly detection module is assessed for its ability to distinguish between malicious activity, such as DDoS attacks or intrusion attempts, and regular system deviations. Automated recovery actions are tested to ensure they do not inadvertently expose vulnerabilities while reallocating resources or migrating services. Regular security audits simulate breach attempts and measure the system's capacity to maintain integrity and continuity. Security testing ensures the self-healing system not only responds to faults autonomously but also protects infrastructure, data, and operations from cyber threats, maintaining confidentiality, integrity, and availability.

APPENDICES

Appendix-1

SOURCE CODE

```
flask import Flask, render_template, request, url_for, send_from_directory, Response, jsonify
from collections import Counter
from ultralytics import YOLO
import os
from werkzeug.utils import secure_filename
from ultralytics.utils.plotting import Annotator
import cv2
import datetime
import time
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import google.generativeai as genai
from google.api_core import exceptions as api_exceptions
import json
from pathlib import Path
import threading
import concurrent.futures
import secrets

# Try to load a local .env file if python-dotenv is installed. This is optional
```

```
# and won't raise if the package isn't available; it simply helps local testing.
try:
    from dotenv import load_dotenv
    load_dotenv()
except Exception:
    pass

app = Flask(__name__)

# Folder configurations
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
REPORT_FOLDER = 'reports'
OUTPUT_FOLDER = 'static/outputs'

# Ensure directories exist
for folder in [UPLOAD_FOLDER, REPORT_FOLDER, OUTPUT_FOLDER]:
    if not os.path.exists(folder):
        os.makedirs(folder)

# Load YOLO model for Indian Sign Language
model_path = "best.pt"  # Hypothetical model file
model = YOLO(model_path)

# Configure Gemini client from environment variable for safety
# Support either GENAI_API_KEY (preferred) or GOOGLE_API_KEY (some SDKs expect
this name)
GENAI_API_KEY = os.getenv('GENAI_API_KEY') or os.getenv('GOOGLE_API_KEY')
if GENAI_API_KEY:
```

```
    # Ensure SDK sees GOOGLE_API_KEY env var as some libraries expect that name
    os.environ.setdefault('GOOGLE_API_KEY', GENAI_API_KEY)
    try:
        genai.configure(api_key=GENAI_API_KEY)
    except Exception as e:
        print(f"Warning: failed to configure google.generativeai: {e}")
else:
    print(
        "Warning: no Gemini API key found. Set GENAI_API_KEY or GOOGLE_API_KEY, or
configure ADC. "
        "Gemini calls will be skipped or fallback will be used."
    )

# Simple in-memory circuit breaker for Gemini calls
GEMINI_DISABLED_UNTIL = 0
GEMINI_COOLDOWN_SECONDS = 300  # cooldown when quota errors occur (seconds)

# Persistent state to survive process restarts (stores disabled_until)
GEMINI_STATE_PATH = Path(__file__).parent / '.gemini_state.json'
_gemini_state_lock = threading.Lock()

# Admin token file for protecting runtime key set endpoint
ADMIN_TOKEN_PATH = Path(__file__).parent / '.admin_token'
ADMIN_TOKEN = None

def _init_admin_token():
    global ADMIN_TOKEN
    # Priority: environment variable (preconfigured) -> existing file -> generate new token
    env_token = os.getenv('ADMIN_TOKEN')
```

```
    if env_token:
        ADMIN_TOKEN = env_token
        return
    try:
        if ADMIN_TOKEN_PATH.exists():
            ADMIN_TOKEN = ADMIN_TOKEN_PATH.read_text(encoding='utf-8').strip()
            if ADMIN_TOKEN:
                return
    except Exception:
        pass
    # generate a strong random token and persist it
    ADMIN_TOKEN = secrets.token_urlsafe(24)
    try:
        ADMIN_TOKEN_PATH.write_text(ADMIN_TOKEN, encoding='utf-8')
        print(f"Admin token generated and saved to {ADMIN_TOKEN_PATH}. Token (showing
once): {ADMIN_TOKEN}")
    except Exception:
        print("Warning: failed to persist admin token to file; token printed above only.")


# initialize admin token at startup
_init_admin_token()


def _load_gemini_state():
    global GEMINI_DISABLED_UNTIL
    try:
        if GEMINI_STATE_PATH.exists():
            with GEMINI_STATE_PATH.open('r', encoding='utf-8') as f:
                data = json.load(f)
            GEMINI_DISABLED_UNTIL = float(data.get('disabled_until', 0))
```

```
    except Exception:
        # ignore errors reading state
        GEMINI_DISABLED_UNTIL = GEMINI_DISABLED_UNTIL
import os
import sys

def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'A2SL.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

def _save_gemini_state():
    try:
        data = {'disabled_until': GEMINI_DISABLED_UNTIL}
        with _gemini_state_lock:
            with GEMINI_STATE_PATH.open('w', encoding='utf-8') as f:
                json.dump(data, f)
    except Exception:
```

```
        # ignore write errors
        pass

# load persisted state at startup
_load_gemini_state()

# Email configurations
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
EMAIL_FROM = 'your-email@gmail.com'
EMAIL_PASSWORD = 'your-email-password'
EMAIL_TO = 'prosdgunal@gmail.com'

def send_email_alert(sign_list, report_filename, report_path):
    """Send an email alert with detection results and report attachment."""
    try:
        msg = MIMEMultipart()
        msg['From'] = EMAIL_FROM
        msg['To'] = EMAIL_TO
        msg['Subject'] = 'Indian Sign Language Detection Alert'

        body = f"""
        Indian Sign Language Activity Detected!

        Detected Signs: {', '.join(sign_list)}

        A detailed report is attached for your reference.
        """
```

```
        msg.attach(MIMEText(body, 'plain'))

        with open(report_path, 'rb') as attachment:
            part = MIMEBase('application', 'octet-stream')
            part.set_payload(attachment.read())

        encoders.encode_base64(part)
        part.add_header(
            'Content-Disposition',
            f'attachment; filename= {report_filename}'
        )
        msg.attach(part)

        server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
        server.starttls()
        server.login(EMAIL_FROM, EMAIL_PASSWORD)
        server.sendmail(EMAIL_FROM, EMAIL_TO, msg.as_string())
        server.quit()
        print("Email alert sent successfully!")
    except Exception as e:
        print(f"Error sending email: {e}")

def process_image(results, model, image_path):
    """Process image detection results and save annotated image."""
    output_path = os.path.join(OUTPUT_FOLDER, 'output_image.jpg')
    image = cv2.imread(image_path)
    for r in results:
        annotator = Annotator(image)
```

```
        boxes = r.boxes
        for box in boxes:
            b = box.xyxy[0]
            c = box.cls
            annotator.box_label(b, model.names[int(c)])
        img = annotator.result()
        cv2.imwrite(output_path, img)
    print(f"Image saved to {output_path}")
    return 'outputs/output_image.jpg'

def process_video(video_path, model):
    """Process video detection results and save annotated video with reduced FPS."""
    output_path = os.path.join(OUTPUT_FOLDER, 'output_video.mp4')
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print("Error: Could not open video file")
        return None

    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    input_fps = int(cap.get(cv2.CAP_PROP_FPS))
    output_fps = 5  # Set reduced FPS for slower video processing

    fourcc = cv2.VideoWriter_fourcc(*'H264')
    out = cv2.VideoWriter(output_path, fourcc, output_fps, (frame_width, frame_height))

    sign_list = []
    frame_count = 0
```

```
frame_interval = max(1, int(input_fps / output_fps))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    if frame_count % frame_interval != 0:
        continue

    results = model.predict(frame)
    annotator = Annotator(frame)
    for r in results:
        boxes = r.boxes
        for box in boxes:
            b = box.xyxy[0]
            c = box.cls
            annotator.box_label(b, model.names[int(c)])
            sign_list.append(model.names[int(c)])

    annotated_frame = annotator.result()
    out.write(annotated_frame)

cap.release()
out.release()
print(f"Video saved to {output_path}")
return 'outputs/output_video.mp4', list(set(sign_list))
```

```
def analyze_with_gemini(image_path, sign_list=None, max_retries=3, initial_backoff=1.0):
    """Analyze image using Gemini-2.0-flash model with retry/backoff and graceful fallback.

    This function implements:
    - a simple in-memory cooldown/circuit-breaker when quota is exhausted,
    - exponential backoff retries for transient errors,
    - a safe fallback string when analysis cannot be obtained.
    """
    global GEMINI_DISABLED_UNTIL

    # If we previously detected quota exhaustion, skip calling Gemini until cooldown expires
    if time.time() < GEMINI_DISABLED_UNTIL:
        return "Gemini analysis temporarily disabled due to quota/exhaustion."

    # If no API key was configured and no ADC, avoid calling the SDK (it will raise).
    # Provide a helpful message and local fallback using detected labels if available.
    if not GENAI_API_KEY:
        msg = (
            "Gemini analysis failed: No API key or ADC found. "
            "Set the environment variable `GENAI_API_KEY` or `GOOGLE_API_KEY`, or
configure Application Default Credentials (ADC)."
        )
        if sign_list:
            msg += f" Local fallback summary: Detected signs: {', '.join(sign_list)}."
        else:
            msg += " Local fallback summary: No detected signs available."
        return msg
```

```
    # Do NOT send local file paths to the remote Gemini model (it cannot access local files).
    # Instead, if we have detected sign labels, ask Gemini to interpret those labels
    # and infer context/meaning. If no labels are available, request general guidance.
    if sign_list:
        prompt = (
            "Interpret the following detected Indian Sign Language labels and infer likely meaning, "
            f"grammatical role, and possible contexts: {', '.join(sign_list)}. "
            "Note: the actual image file is not accessible; use only the labels provided. "
            "Provide possible translations, usage notes, and suggestions for verifying the
interpretation."
        )
    else:
        prompt = (
            "I cannot access the image (it is on a local filesystem). "
            "Please provide general guidance on how to analyze an Indian Sign Language image: "
            "what handshapes, locations, movements, and facial markers to look for, and "
            "questions to ask the user to help disambiguate signs when the image cannot be shared."
        )

    # Use a thread executor to enforce per-call timeouts (protects the request thread
    # from hanging if the remote API is unresponsive). We'll do up to `max_retries` attempts
    # but cap the total waiting time so user requests don't block indefinitely.
    backoff = initial_backoff
    max_total_wait = 15.0  # seconds total waiting across retries
    total_wait = 0.0
    per_call_timeout = 5.0  # seconds to wait for each SDK call

    def _call_genai(prompt_text):
        # This function runs in worker thread and calls the SDK.
```

```
    if hasattr(genai, 'GenerativeModel'):
        model_obj = genai.GenerativeModel('gemini-2.5-flash-lite')
        return model_obj.generate_content(prompt_text)
    if hasattr(genai, 'generate'):
        return genai.generate(model='gemini-2.5-flash-lite', input=prompt_text)
    if hasattr(genai, 'generate_text'):
        return genai.generate_text(model='gemini-2.5-flash-lite', prompt=prompt_text)
    raise RuntimeError('No supported google.generativeai call available')

with concurrent.futures.ThreadPoolExecutor(max_workers=1) as executor:
    for attempt in range(1, max_retries + 1):
        # Check cooldown state (may have been set by previous runs)
        if time.time() < GEMINI_DISABLED_UNTIL:
            return "Gemini analysis temporarily disabled due to quota/exhaustion."

        future = executor.submit(_call_genai, prompt)
        try:
            response = future.result(timeout=per_call_timeout)
            if hasattr(response, 'text'):
                return response.text
            return str(response)

        except concurrent.futures.TimeoutError:
            future.cancel()
            err = f"Gemini call timed out on attempt {attempt} (timeout={per_call_timeout}s)"
            print(err)
            total_wait += per_call_timeout
        except api_exceptions.ResourceExhausted as e:
```

```
            print(f"Gemini quota exhausted (attempt {attempt}): {e}")
            GEMINI_DISABLED_UNTIL = time.time() + GEMINI_COOLDOWN_SECONDS
            _save_gemini_state()
            return "Gemini analysis unavailable due to API quota limits."
        except Exception as e:
            print(f"Gemini API error on attempt {attempt}: {e}")
            total_wait += per_call_timeout

        # Decide whether to continue retrying
        if total_wait >= max_total_wait or attempt == max_retries:
            # Set cooldown to avoid repeated failures hammering the API
            GEMINI_DISABLED_UNTIL = time.time() + GEMINI_COOLDOWN_SECONDS
            _save_gemini_state()
            fallback = f"Gemini analysis failed after {attempt} attempts/timeouts."
            if sign_list:
                fallback += f" Local fallback summary: Detected signs: {', '.join(sign_list)}."
            else:
                fallback += " Local fallback summary: No detected signs available."
            return fallback

        # Sleep with exponential backoff but ensure we don't exceed max_total_wait
        sleep_time = min(backoff, max_total_wait - total_wait)
        time.sleep(sleep_time)
        total_wait += sleep_time
        backoff *= 2

def run_object_detection(file_path, is_video=False):
    """Run object detection on image or video."""
```

```
    if is_video:
        output_path, sign_list = process_video(file_path, model)
        print("Detected signs:", sign_list)
    else:
        results = model.predict(file_path)
        sign_counts = Counter(model.names[int(c)] for r in results for c in r.boxes.cls)
        sign_list = list(sign_counts.keys())
        print("Detected signs:", sign_list)
        output_path = process_image(results, model, file_path)

    gemini_analysis = analyze_with_gemini(file_path, sign_list=sign_list)
    print("Gemini Analysis:", gemini_analysis)

    return sign_list, output_path, gemini_analysis

def gen_frames():
    """Generate frames for live camera feed."""
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        results = model.predict(frame)
        for box in results[0].boxes:
```

```
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            label = results[0].names[int(box.cls[0])]
            confidence = round(box.conf[0].item(), 2)
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame, f"{label} {confidence}", (x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        ret, buffer = cv2.imencode('.jpg', frame)
        frame = buffer.tobytes()
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

    cap.release()

@app.route('/')
def landing():
    """Render landing page."""
    return render_template('landing.html')

@app.route('/index')
def upload():
    """Render upload page."""
    return render_template('index.html')

@app.route('/live')
def live():
    """Render live camera page."""
    return render_template('live.html')
```

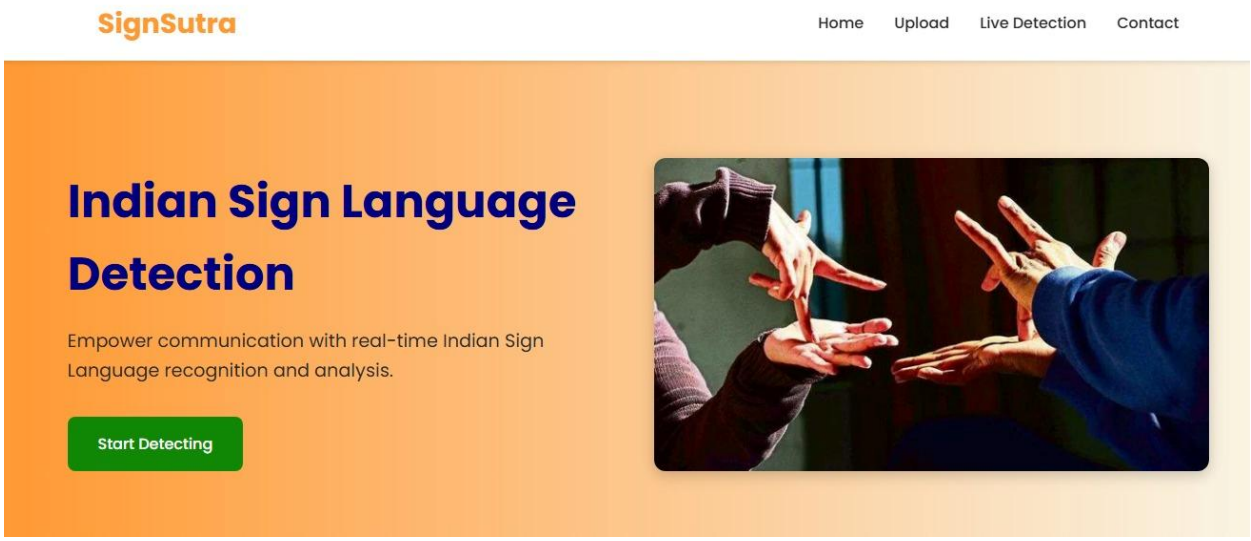
```
@app.route('/video_feed')
        found = False
       for i, line in enumerate(lines):
            if line.strip().startswith(f"{var_name}="):
                lines[i] = f"{var_name}={var_value}"
                found = True
                break
        if not found:
            lines.append(f"{var_name}={var_value}")
        env_path.write_text('\n'.join(lines) + '\n', encoding='utf-8')
        return True
    except Exception as e:
        print(f"Warning: failed to write .env file: {e}")
        return Fals
@app.route('/admin/set_genai_key', methods=['POST'])
def admin_set_genai_key():
    """Admin endpoint to set/persist GENAI_API_KEY at runtime.
    Expects JSON: {"key": "<API_KEY>", "token": "<ADMIN_TOKEN>"}
    The endpoint updates the running process environment, calls genai.configure,
    and persists the key to `.env` for future restarts.
    """
    data = request.get_json(silent=True) or {}
    key = data.get('key')
```

```
    token = data.get('token')
    if not key or not token:
        return jsonify({'ok': False, 'error': 'Missing key or token in request (JSON).'}), 400
    # validate token
    if ADMIN_TOKEN is None or token != ADMIN_TOKEN:
        return jsonify({'ok': False, 'error': 'Invalid admin token.'}), 403
    # persist to .env and set runtime environment
    ok = _update_env_file('GENAI_API_KEY', key)
    if not ok:
        return jsonify({'ok': False, 'error': 'Failed to persist .env file.'}), 500
    os.environ['GENAI_API_KEY'] = key
    os.environ['GOOGLE_API_KEY'] = key
    try:
        genai.configure(api_key=key)
    except Exception as e:
        return jsonify({'ok': False, 'error': f'Failed to configure genai: {e}'}), 500
    return jsonify({'ok': True, 'message': 'GENAI_API_KEY updated and persisted.'})
if __name__ == '__main__':
    app.run(debug=True)
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys
```

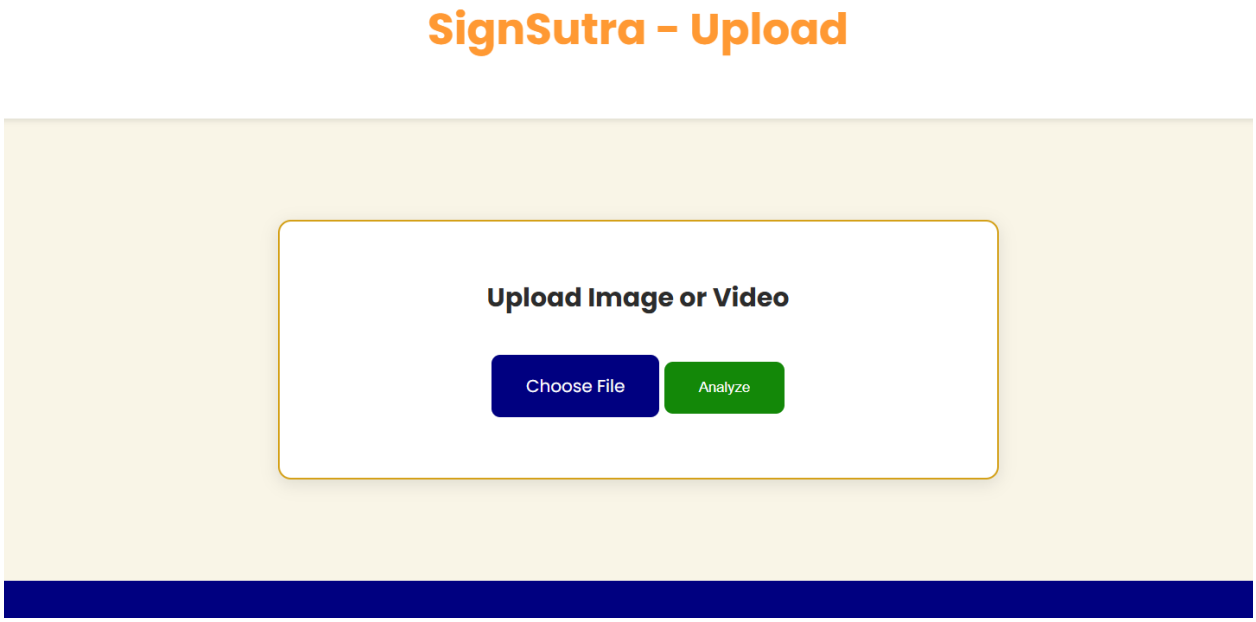
```
def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'A2SL.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

Appendix-2

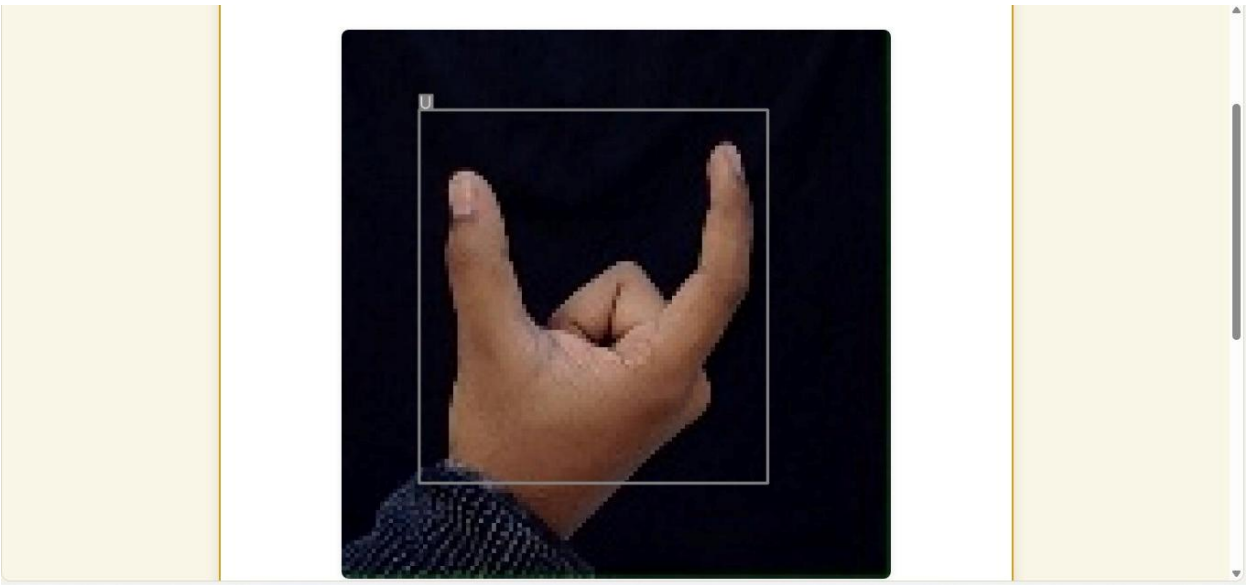
1.Sign Sutra



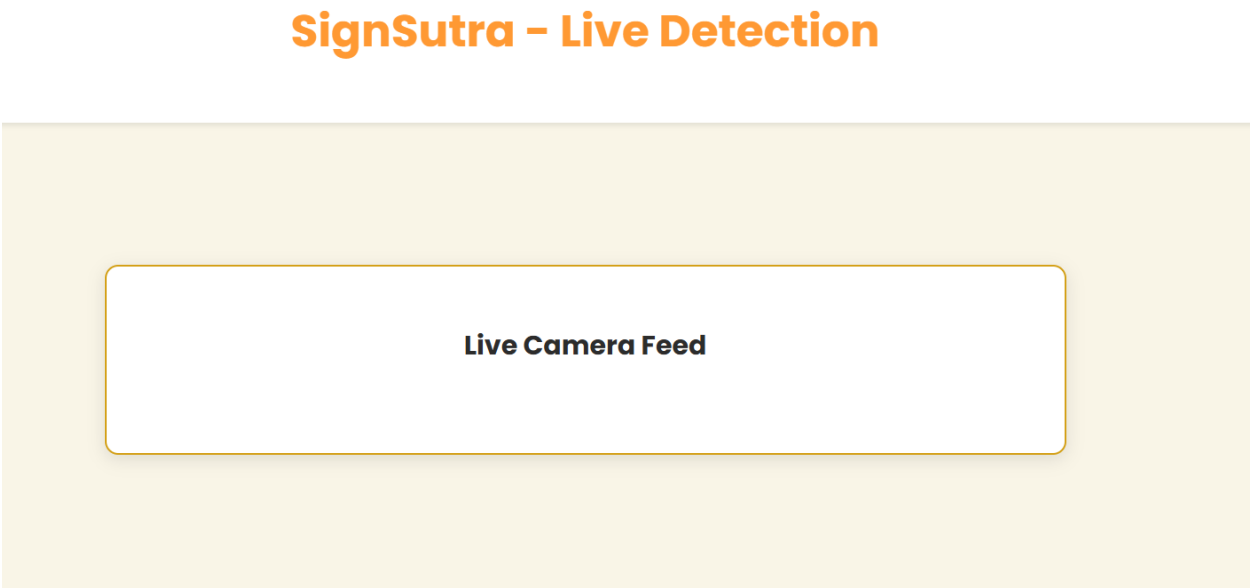
Home Page



ISL-Upload image

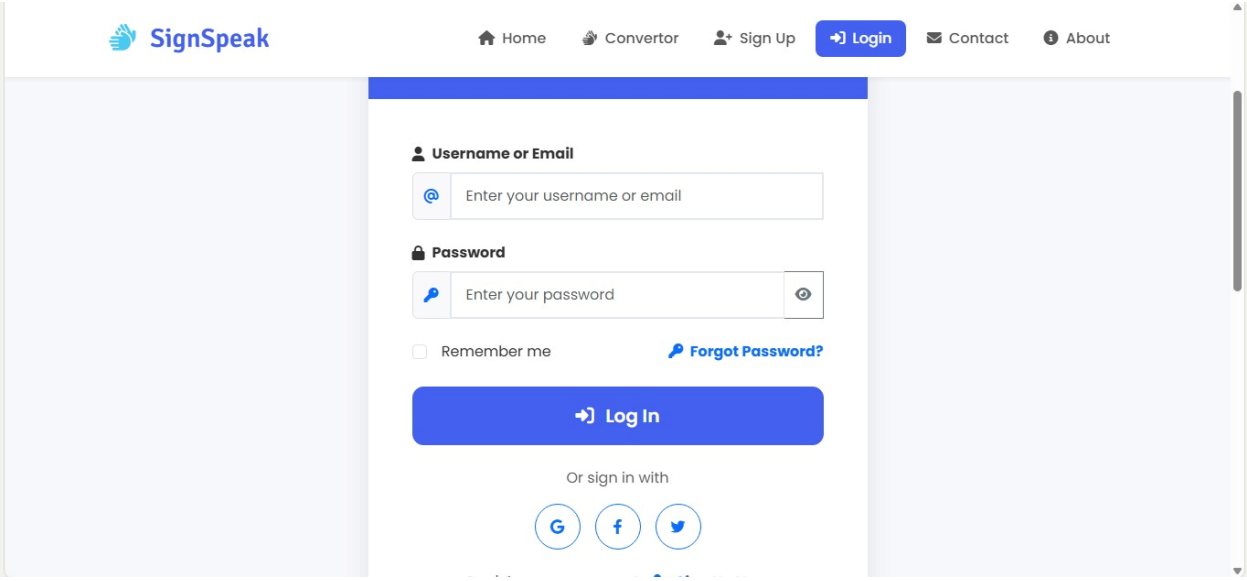


Output

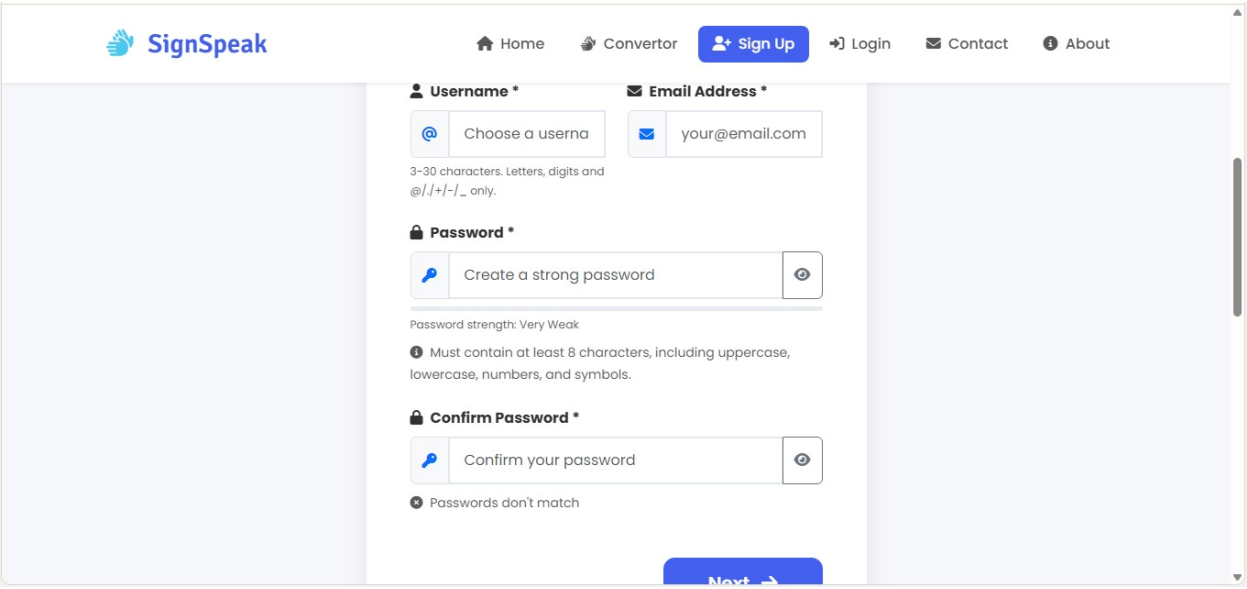


Live Detection

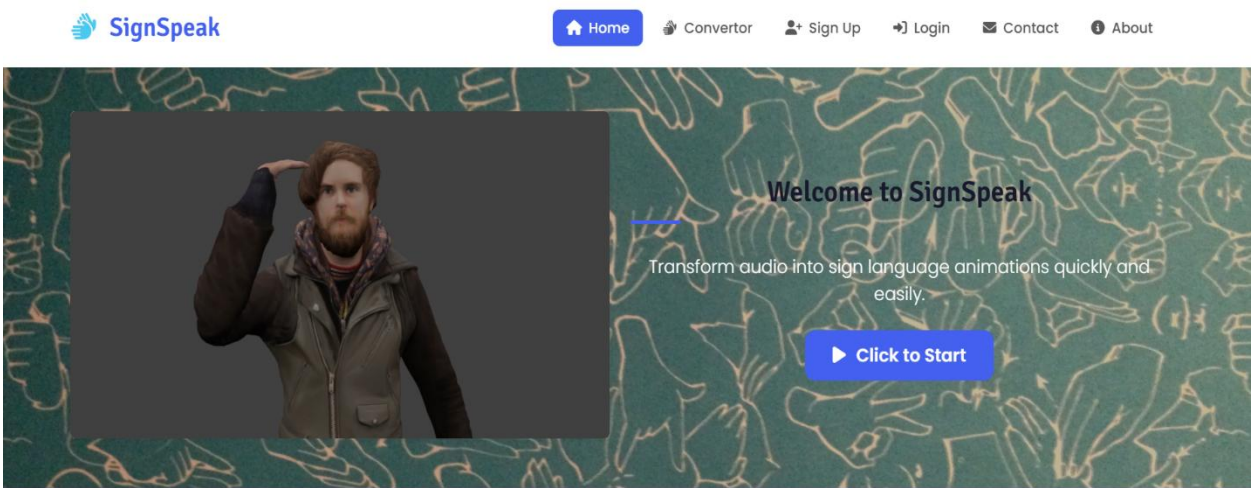
2.Sign Speak



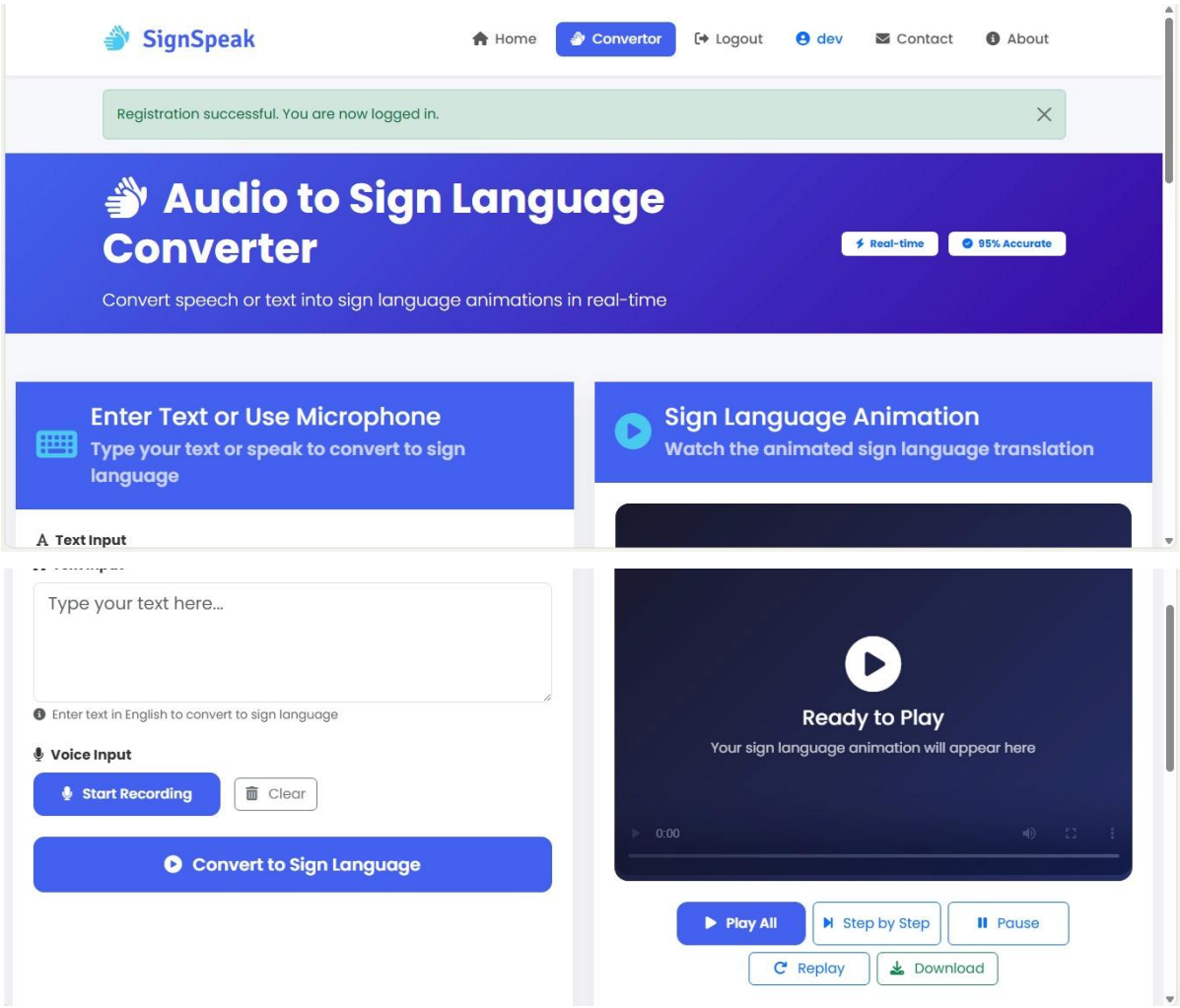
Login Page



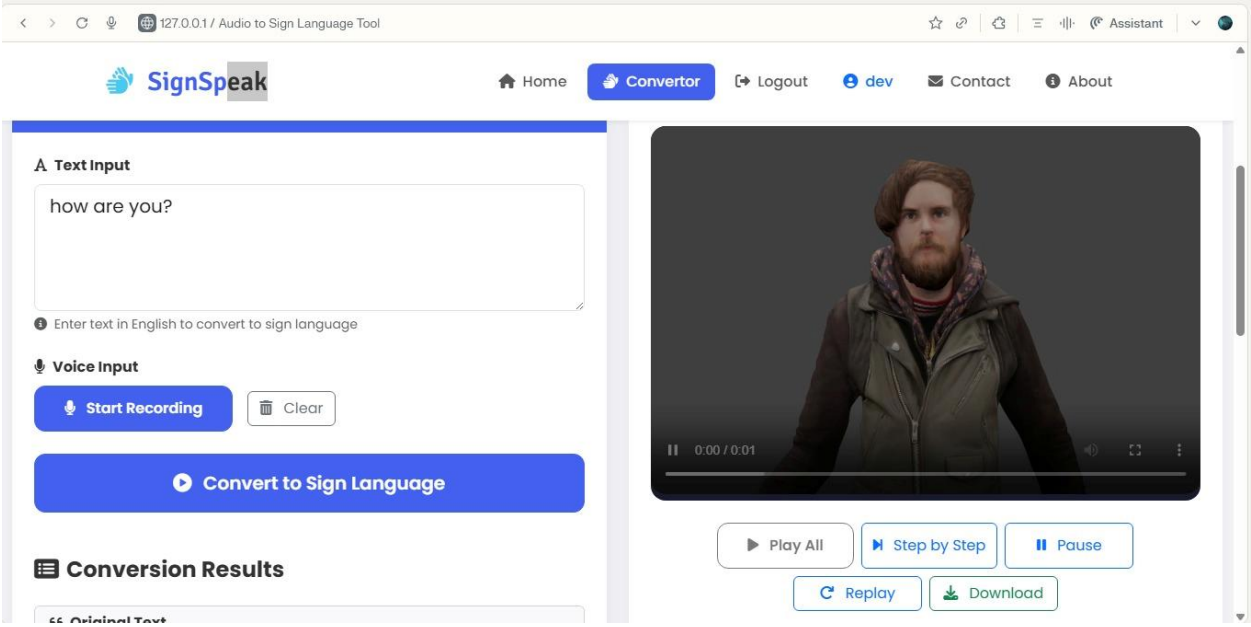
Sign Up Page



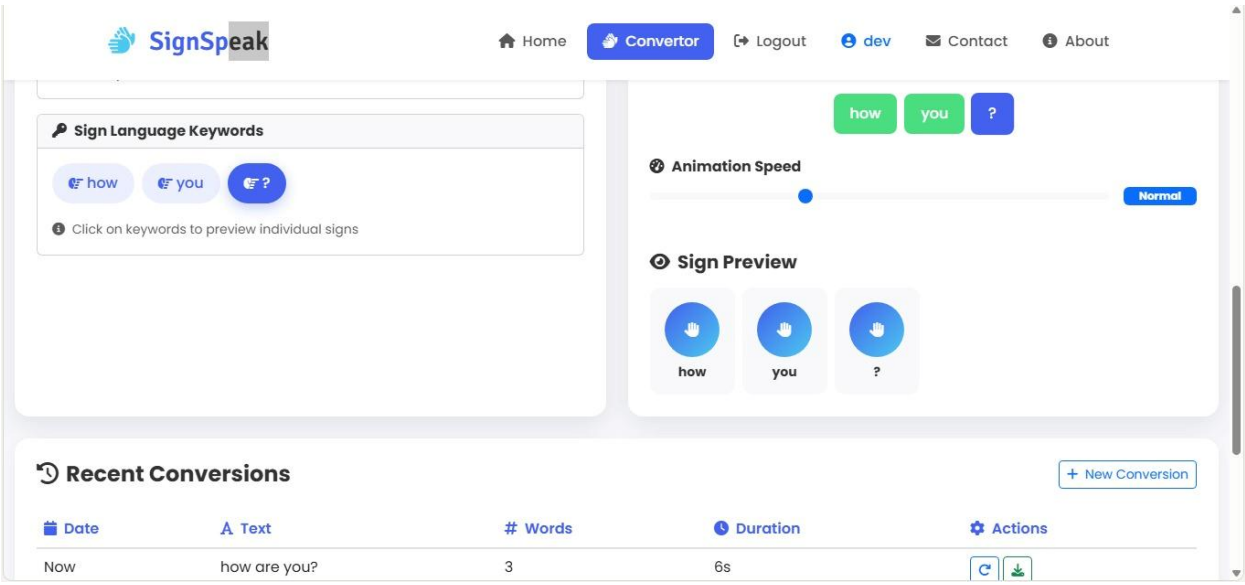
Home page



ISL-Converter



Visual point-1



Visual point-2

CONCLUSION AND FUTURE ENHANCEMENT

1. CONCLUSION

The proposed real-time bidirectional translation system demonstrates a significant advancement in **inclusive communication technology**, bridging interactions between hearing individuals and those who are deaf or speech-impaired. By combining **computer vision, deep learning, speech recognition, and virtual avatar technologies**, the system enables seamless translation from **sign language to speech** and from **speech to animated signs**. The implementation of **Convolutional Neural Networks** for gesture recognition and **automatic speech recognition** for speech processing ensures high accuracy and low latency, essential for natural, real-time interaction.

The integration of a **virtual avatar** to render gestures provides an intuitive visual representation, enhancing comprehension for both users and observers. The system's design emphasizes **scalability, adaptability, and usability**, supporting multiple sign languages and spoken languages with minimal retraining. The interactive interface ensures accessibility, allowing users to adjust preferences and receive immediate textual or auditory feedback.

This work illustrates the practical application of **artificial intelligence in assistive technology**, highlighting its potential to reduce communication barriers, foster social inclusion, and improve quality of life for individuals with hearing and speech impairments. The combination of **gesture recognition, speech synthesis, and multimodal visualization** demonstrates a model of effective human-computer interaction that can be applied in educational, professional, and public service environments.

Overall, the system not only validates the feasibility of real-time sign-speech translation but also establishes a framework for future developments in **inclusive AI solutions**. By integrating advanced machine learning models with intuitive visualization techniques, the project reinforces the value of technology-driven accessibility and promotes equitable participation in communication across diverse communities.

2. FUTURE ENHANCEMENT

While the current system achieves **real-time bidirectional translation**, several enhancements can further improve performance, usability, and global applicability. One key improvement involves the integration of **multilingual support** for both spoken and sign languages, enabling seamless communication across regions and cultures. Leveraging **transfer learning** and **modular gesture datasets**, the system can adapt to multiple dialects and regional variations, expanding its accessibility.

Another enhancement includes incorporating **context-aware language models** to improve the accuracy of gesture-to-text and speech-to-gesture mapping. By understanding semantic context and sentence structure, the system can handle complex phrases, idiomatic expressions, and conversational nuances, ensuring more natural communication. Additionally, **emotion recognition** from facial expressions and vocal tone could be integrated, allowing the virtual avatar to reflect not only words but also emotional content, enhancing engagement and understanding.

Optimizing the system for **mobile and wearable devices** is another significant direction, enabling on-the-go interaction without reliance on desktop setups. Edge computing and lightweight neural networks can maintain **low-latency processing** while minimizing computational requirements. Furthermore, the inclusion of **adaptive user interfaces** tailored to individual needs, such as adjustable avatar speed, gesture size, or text display, can enhance usability for diverse user groups.

Finally, expanding the system to support **collaborative and group communication** scenarios can facilitate real-world applications in classrooms, workplaces, and public spaces. By continuously updating the gesture and speech databases through **community-driven learning**, the system can evolve to address emerging communication patterns and maintain cultural relevance. These enhancements will strengthen the system's role as a comprehensive **assistive technology platform**, promoting inclusivity, accessibility, and equitable human-computer interaction on a global scale.

BIBLIOGRAPHY

1. *V. C. Handaragall and J. SubhashMetan, "Comprehensive Analysis of Deep Fake Detection Techniques for Images and Videos using Deep Learning," 2025 International Conference on (ICICKE), Bengaluru, India, 2025, pp. 1-5, doi: 10.1109/ICICKE65317.2025.11136286.*

2. *Gurjar, L. Bopche, A. Choudhary and K. Khurana, "Fake Image Detection Using Biomarkers with Deep Neural Network," 2025 IEEE (ICWITE), Bangalore, India, 2025, pp. 1-6, doi: 10.1109/ICWITE64848.2025.11307094.*

3. *P. Shetty, D. K. R and D. S. Padre, "Deepfake Detection using Hybrid Deep Learning: Enhancing Accuracy with ResNet, GANs, and Attention Mechanisms," 2025 (ICAISS), Trichy, India, 2025, pp. 95-99, doi: 10.1109/ICAISS61471.2025.11041976.*

4. *U. Narayan, A. Kumar and K. Kumar, "Fake News Detection using Hybrid of Deep Neural Network and Stacked LSTM," 2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 2021, pp. 385-390, doi: 10.1109/ICAC3N53548.2021.9725519.*

5. *K. Bansal, S. Agarwal and N. Vyas, "Deepfake Detection Using CNN and DCGANS to Drop-Out Fake Multimedia Content: A Hybrid Approach," 2023 International Conference on IoT, Communication and Automation Technology (ICICAT), Gorakhpur, India, 2023, pp. 1-6, doi: 10.1109/ICICAT57735.2023.10263628.*

6. *D. Prabakar, R. Ganesan, D. L. Rani, P. Neti, N. Kalyani and S. K. Mudradi, "Hybrid Deep Learning Model for Copy Move Image Forgery Detection," 2022 Sixth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), Dharan, Nepal, 2022, pp. 1023-1028, doi: 10.1109/I-SMAC55078.2022.9987319.*

7. *S. I. Rahmani, T. Pal, V. Chaudhary and M. B. Nirmohi, "Comparative Analysis of Deep-Fake Detection Methods," 2025 3rd International Conference on Disruptive Technologies (ICDT), Greater Noida, India, 2025, pp. 651-654, doi: 10.1109/ICDT63985.2025.10986625.*

8. *Swathi, P. Priyanka, S. Sneha, V. R. Banka, G. Sagar and R. Shaik, "Enhanced Deep Fake Detection Using Deep Learning," 2025 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), Bhubaneswar, India, 2025, pp. 1-6, doi: 10.1109/ASSIC64892.2025.11158181.*

9. *Abbas, N. A. Helal, W. Gad and A. Hamad, "A Survey on Fake Image Detection Methodologies: Trends and Techniques," 2025 Twelfth International Conference on Intelligent Computing and Information Systems (ICICIS), Cairo, Egypt, 2025, pp. 359-364, doi: 10.1109/ICICIS66182.2025.11313172.*

10. *Ramarao, J. Nagaraju, J. Thati and K. Gopi, "Hybrid CNN-Transformer Model with Multi-Frequency Analysis for Robust Fake Image Detection," 2025 3rd International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI), Coimbatore, India, 2025, pp. 74-80, doi: 10.1109/ICoICI65217.2025.11252554.*